

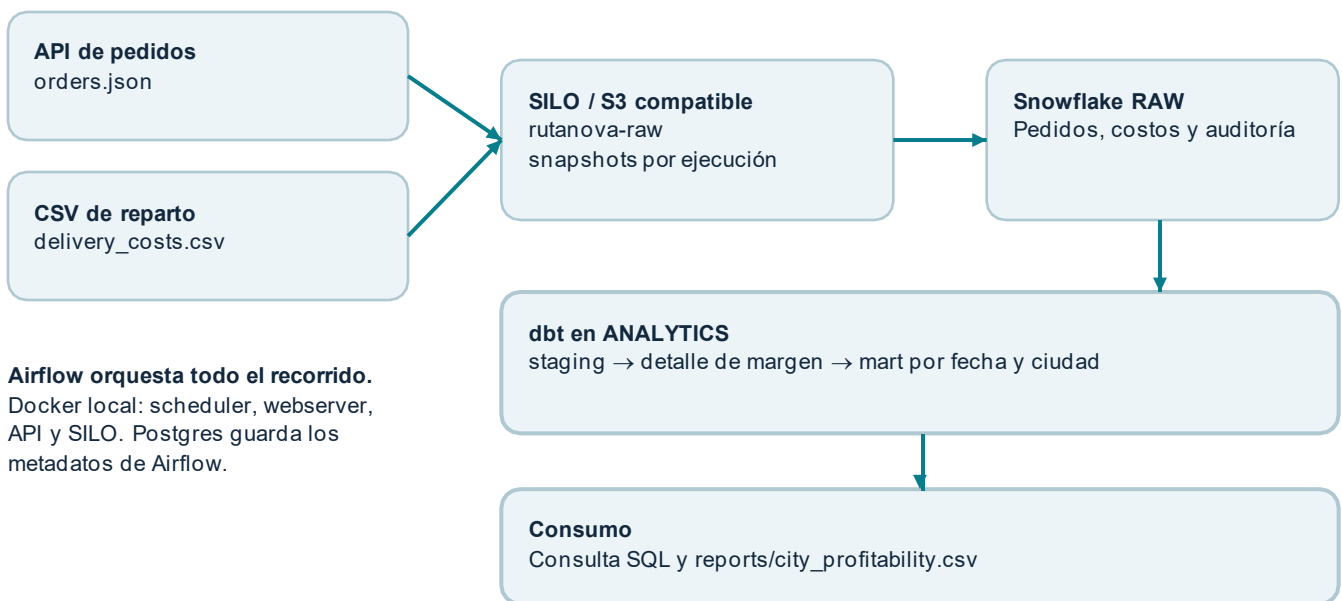
RutaNova

Rentabilidad diaria de pedidos entregados por ciudad

01 Caso de negocio

¿Qué ciudad deja más margen después del costo del producto y del reparto? RutaNova integra una API HTTP de pedidos y un CSV del transportista. Ambas fuentes son **simuladas y reproducibles**; el pipeline y Snowflake se ejecutan realmente. Se excluyen cancelados. El margen de contribución no incluye impuestos ni gastos fijos.

02 Arquitectura y recorrido del dato



03 Decisiones de diseño

Orquestación: DAG diario a las 07:00 de Lima, catchup=False, max_active_runs=1, TaskGroups y PythonSensor en reschedule. Retries con backoff, owner y doc_md.

Carga: snapshot completo con DELETE + INSERT en transacción; rollback ante fallos. Los originales quedan en SILO. DDL fuera de la transacción; sin CDC.

Modelado: 4 modelos encadenados. Margen = ingreso - producto - reparto; porcentaje ponderado por ingreso. dbt build se lanza desde PythonOperator (sin Cosmos).

Seguridad: Connections cifradas en Airflow, profiles.yml con env_var(), clave privada fuera de Git e imagen. Usuario RUTANOVA_SVC y rol limitado RUTANOVA_ROLE; XSMALL con AUTO_SUSPEND=60.

RutaNova

Validaciones, ajustes y entrega reproducible

04 Evidencia de ejecución real

Airflow: Todas las tareas de la ejecución mostrada en success.

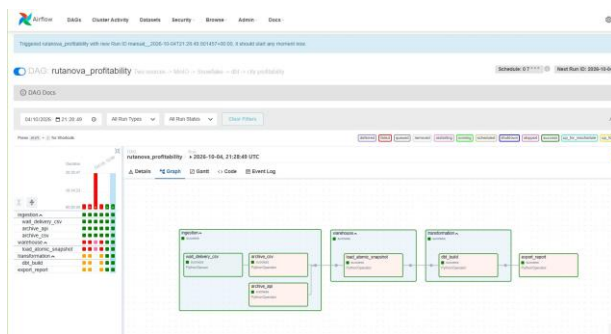
Almacenamiento: Bucket privado; orders.json y delivery_costs.csv por snapshot.

pytest: 15 passed in 2.03s; reglas de negocio y estructura del DAG.

dbt: 4 modelos + 20 tests; PASS=24, WARN=0, ERROR=0, SKIP=0.

Snowflake: Mart visible con 9 filas: 3 fechas por Lima, Arequipa y Trujillo.

Fixture: 18 pedidos, 15 entregados y 3 cancelados. Totales de referencia: **\$/ 2,850 de ingresos y \$/ 779 de margen**; verificar con sql/02_validate_results.sql. Las fechas del ejemplo son 01-03/10/2026.



Airflow: tareas completas en success

The screenshot shows the Snowflake query results for the 'mart' table. The table has 9 rows and 8 columns: ID, DATE, CITY, DELIVERED_ORDERS, TOTAL_REVENUE, TOTAL_PRODUCT_COST, TOTAL_DELIVERY_COST, and CANTIDAD. The data is grouped by date and city.

ID	DATE	CITY	DELIVERED_ORDERS	TOTAL_REVENUE	TOTAL_PRODUCT_COST	TOTAL_DELIVERY_COST	CANTIDAD
1	2026-10-01	Arequipa	2	370.00	160.00	30.00	20
2	2026-10-01	Lima	2	200.00	100.00	20.00	20
3	2026-10-01	Trujillo	1	120.00	60.00	10.00	10
4	2026-10-02	Arequipa	2	380.00	250.00	30.00	30
5	2026-10-02	Lima	2	370.00	230.00	30.00	30
6	2026-10-02	Trujillo	1	180.00	120.00	10.00	10
7	2026-10-03	Arequipa	2	510.00	337.00	30.00	30
8	2026-10-03	Lima	2	480.00	300.00	30.00	30
9	2026-10-03	Trujillo	1	220.00	160.00	10.00	10

Snowflake: mart por fecha y ciudad

05 Ajustes incorporados

Docker: pgsty/minio:RELEASE.2026-08-04T00-00-00Z (SILO) y clickhouse/minio-mc:RELEASE.2025-04-16T18-13-26Z. Airflow en localhost:8081; consola S3 en localhost:9001.

Snowflake: registro de la clave pública completa correspondiente a la privada del scheduler; creación de warehouse, database, schemas y permisos. Sesión comprobada con usuario, rol, base y warehouse correctos.

Pruebas: COPY de api/ e inbox/ al Dockerfile; reconstrucción y recreación de servicios. Corrige el FileNotFoundError de los datos de prueba.